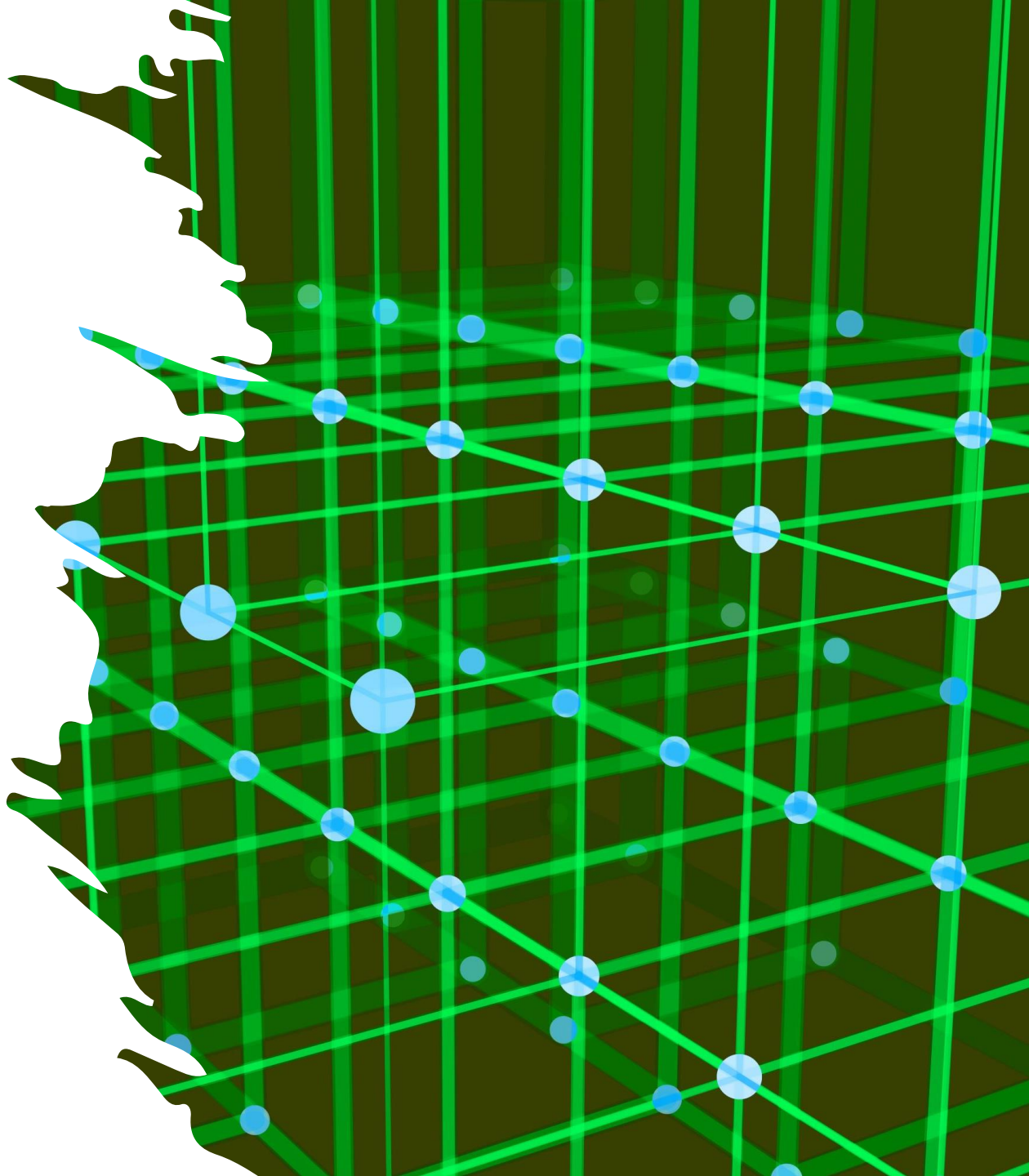
The Master Theorem

Analysis of recursive algorithms
made easy



Recursion

- Calculate a function by calling itself.
- For example, suppose I have a sorted array `arr` and I want to search it for a value `x`.
- ```
  bool find(arr, x)
       if (len(arr) == 1) return (arr[0] == x)
       p := len(arr)/2
       if (arr[p] >= x)
            return find(arr[0:p],x)
       else
            return find(arr[p:len(arr)],x)
  ```
- Correctness?
- Runtime?
  ```

Runtime

- I guess we do a constant amount of work each time the function is called before the next call.
- How many function calls do we have?
- `find(_,n)->find(_,n/2)->find(_,n/4)->…->find(_,n/2^k)->…->find(_,1)`

Sorting

- A sorting algorithm many of you probably know: **mergesort**
- Split the array into two, sort the left half, sort the right half and then merge the two
- Exercise: write pseudocode for the function `merge` to merge two sorted arrays into one (fresh) sorted array
- Idea: `i=j=k=0`

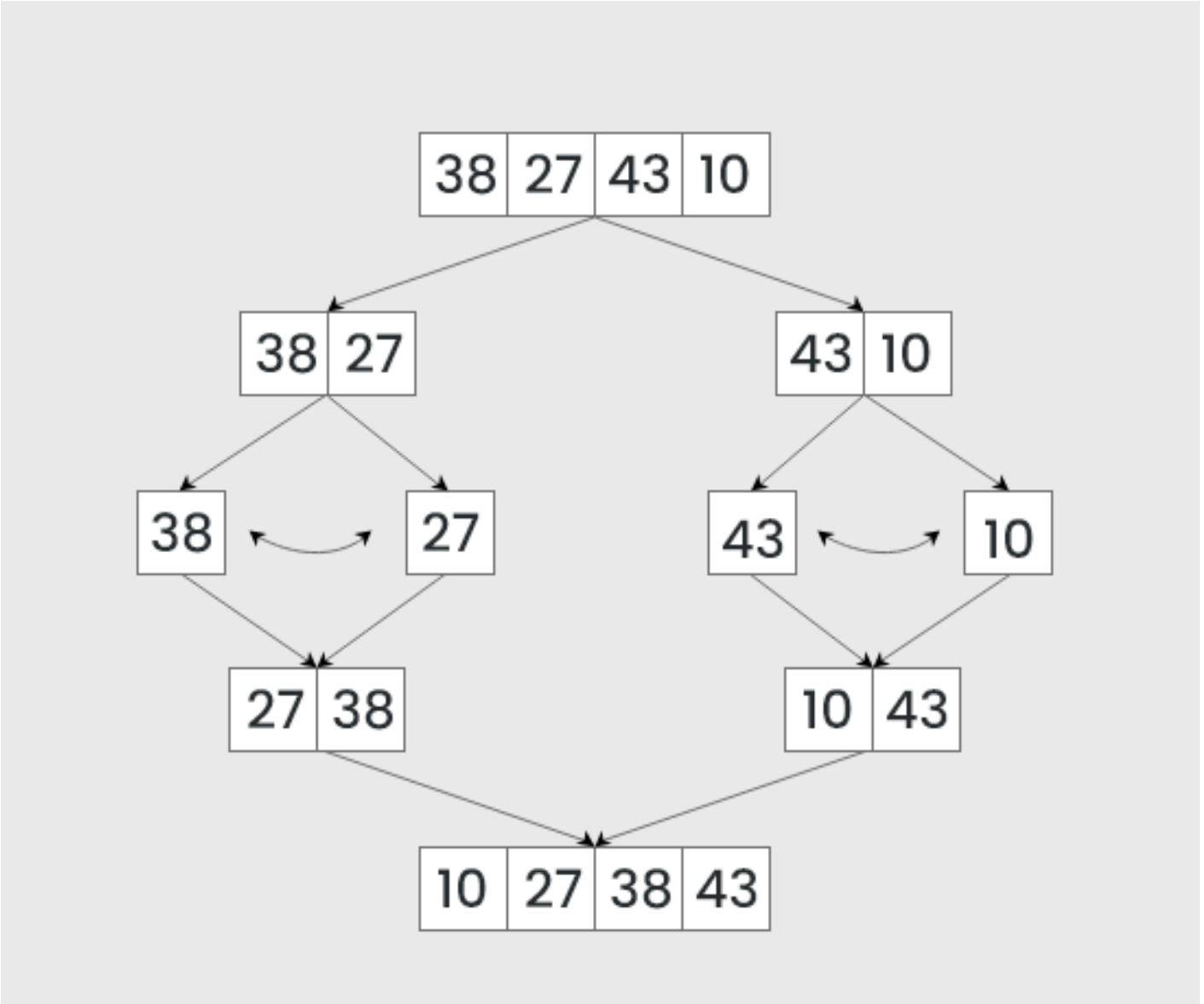
```
while (i < len(arr1)) && (j < len(arr2))
    if (arr1[i] < arr2[j]) ret[k] = arr1[i++]
    else ret[k] = arr2[j++]
    k++
```

Pls don't tell any software engineers I wrote this

- The pseudocode looks like:

```
mergeSort(arr):
      if len(arr) == 1
            return arr
      else
            d = len(arr/2)
            arr1 = mergeSort(arr[0:d])
            arr2 = mergeSort(arr[d:len(arr)])
            return merge(arr1,arr2)
```

Mergesort with 4 elements



What is the complexity of this algorithm?

- Write T(n) for the cost on arrays of length n
- Each recursive call costs T(n/2), merging costs n
- So $T(n) = 2T(n/2) + n$ if $n>1$, $T(1) = 1$
- Unfortunately this is not really an answer :(
- We would like a closed form like $T(n)=n$ or n^2 or something

Solving recurrences

This kind of ‘recurrence relation’ is common for the complexity of recursive algorithms. We have three main options for getting a closed-form solution:

1. Guess
2. Guess with a bit of help – ‘iterative substitution’
3. Draw it out and calculate – the ‘recursion tree’
4. Hit it with a big hammer – the ‘Master Theorem’

Iterative substitution

- We can plug $\frac{n}{2}$ into our formula for T

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n/2}{2}\right) + \frac{n}{2} = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}$$

- Which we can substitute into the original equation

$$T(n) = 2T\left(\frac{n}{2}\right) + n = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 4T\left(\frac{n}{4}\right) + 2n$$

That was fun, let's do that again!

- Plugging in n/4 gives

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n/4}{2}\right) + \frac{n}{4} = 2T\left(\frac{n}{8}\right) + \frac{n}{4}$$

- So we have

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n = 4\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

- Can you spot a pattern?

Iterative substitution

- $T(n) = 2T\left(\frac{n}{2}\right) + n$
- $T(n) = 4T\left(\frac{n}{4}\right) + 2n = 2^2T\left(\frac{n}{2^2}\right) + 2n$
- $T(n) = 8T\left(\frac{n}{8}\right) + 3n = 2^3T\left(\frac{n}{2^3}\right) + 3n$

- Every time we go down one level
  - The exponent on the 2s goes up by one
  - The exponent in the denominator goes up by one
  - As does the constant before the final n
- Write i for the number of times we have recursed

$$T(n) = 2^iT\left(\frac{n}{2^i}\right) + in$$

- This is still kind of a recursion, but fortunately there is one value of T we know

Iterative substitution

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

- We know the answer for T(1)=1, so let's choose i to make $\frac{n}{2^i} = 1$
- We want $n = 2^i$, i.e. $i = \log_2 n$
- $T(n) = nT\left(\frac{n}{n}\right) + n\log n = n + n\log n = O(n\log n)$
- We still need to check that $T(n) = n + n\log n$ is actually a solution:

$$2T\left(\frac{n}{2}\right) + n = 2\left(\frac{n}{2} + \frac{n}{2}\log\frac{n}{2}\right) + n = n + n\log n\ - n + n = n + n\log n = T(n)$$

(remember $\log\frac{n}{2} = \log\ n + \log\frac{1}{2} = \log n\ - 1$)

Guessing

- Actually a legit option, as long as you check your guess works
- I guess I've kind of heard about sorting algorithms being $\mathrm{O}(n \log n)$, so let's try $T(n) = n \log n$
- $2T\left(\frac{n}{2}\right) + n = 2\frac{n}{2}\log\frac{n}{2} + n = n \log n \ - n + n = n \log n$
- Great, but we also need $T(n) = 1$
- Adding 1 doesn't work, but adding n does
- **Then we check** $T(n) = n \log n + n$ **works**, as at the end of the last slide
- (If we wanted to do a little less guessing, we could guess $T(n) = An \log n + Bn + C$ and see what values of A,B and C we need)

The recursion tree

- The idea of the recursion tree is to draw all the calls made by the algorithm in the form of a tree
- Each call is a node in the tree
  - It can be handy to keep some information about the data with each node
- The initial call is the root
- Every recursive call is a child of the call from which it was made
- Calls to the base case are leaf nodes
  - As the algorithm halts, eventually all paths from the root will terminate a leaf node
- The answer can be calculated as

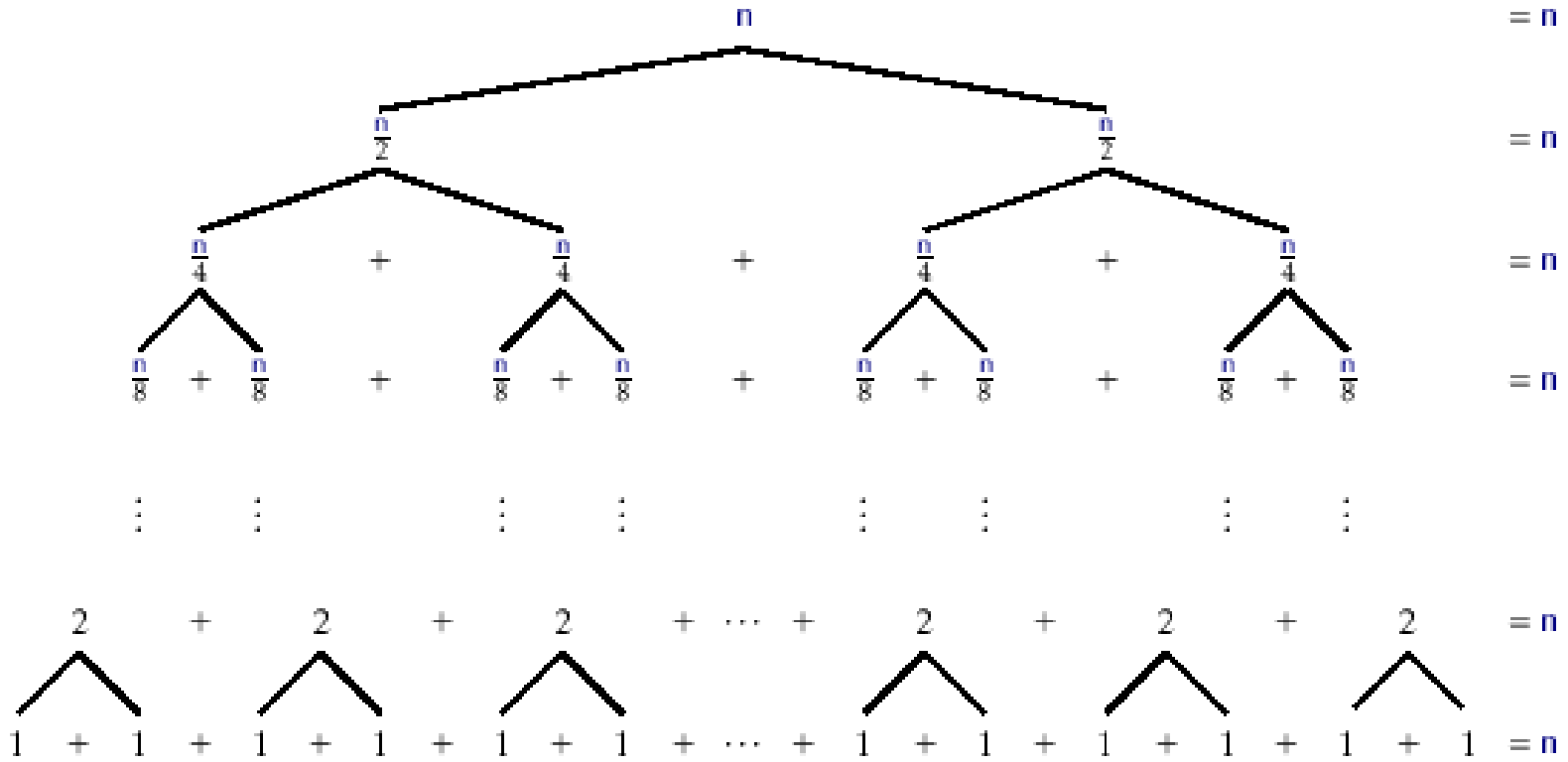
$$number\ of\ nodes\ *cost\ per\ node$$

- Equivalently for a full, balanced tree

$$cost\ per\ level\ *number\ of\ levels$$

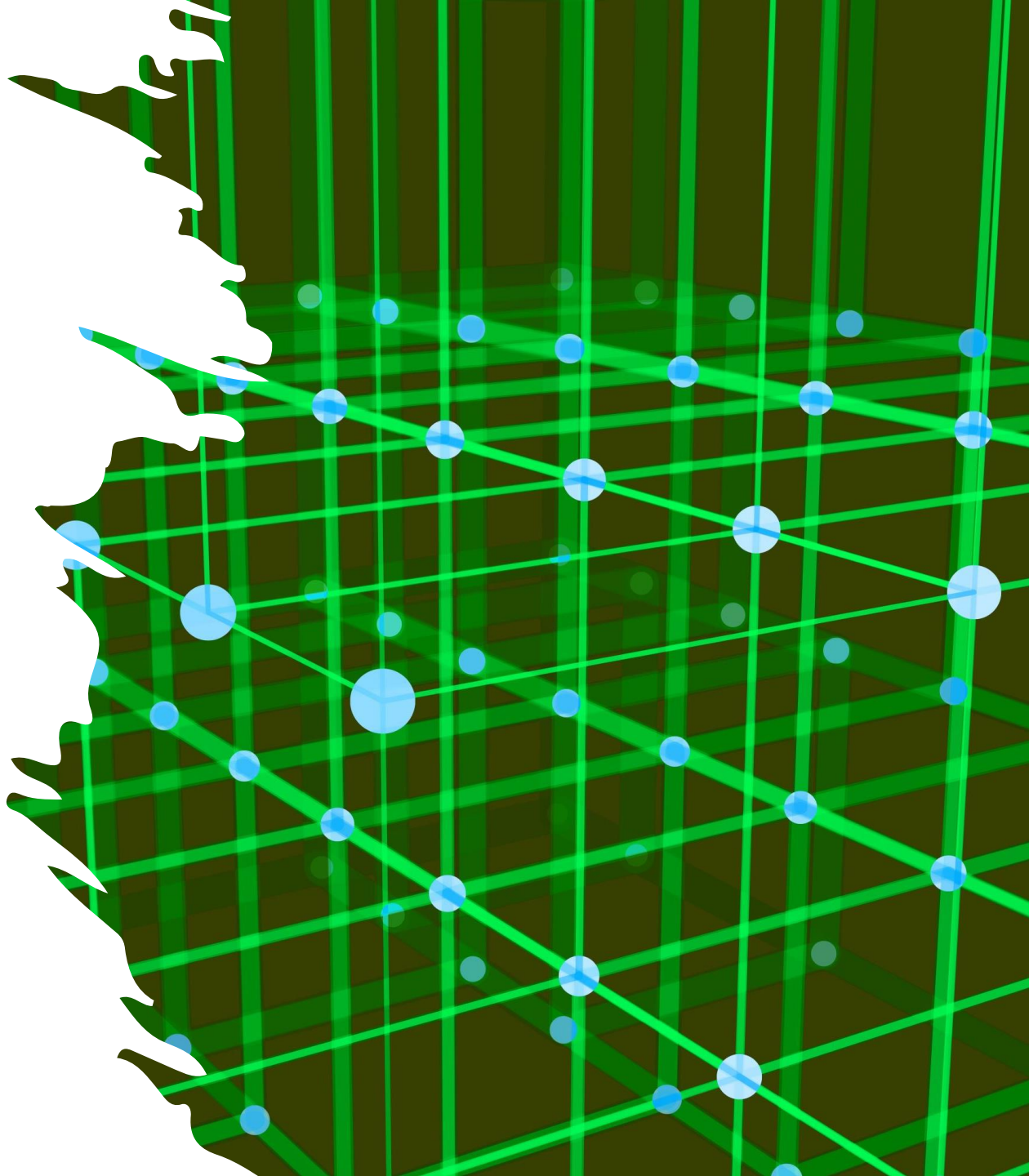
The recursion tree

Here is a recursion tree for mergsort



The Master Theorem

The most important part of this lecture, so pay attention!



The problem

- We want to find the time complexity of a recursive algorithm
- Iterative substitution can be difficult, and not always applicable
- Creating the recursion tree can also be difficult
- Both methods involve a lot of effort and require creativity
  - In addition to being difficult to prove correct
- It would be nice if there were some sort of method that would yield the answer through simple computation

Hmmm

- Let's say I have a recurrence relation of the form

$$T(n) = \begin{cases} 1 & if\ n < d \\ aT\left(\frac{n}{b}\right) + f(n) & if\ n \geq d \end{cases}$$

- (for the moment we'll assume $f(n) = \Theta(n^c)$ for some c)
- Let's forget about $f(n)$ for the moment, so imagine we just have $T(n) = AT(\frac{n}{b})$
- Can we guess a solution?

Hmmm

- OK, looks like $T(n) = \Theta(n^{\log_b a})$ is a solution if $f(n) = 0$
- Intuition: if $f(n)$ is tiny then it's still a solution
- On the other hand if $f(n)$ is humungous then it's way more work than the recursive call and the solution is $\Theta(f(n))$
- The other possiblity is that $f(n)$ is similar to $n^{\log_b a}$

The simple form

- Given an algorithm with a recurrence relation of the form

$$T(n) = \begin{cases} 1 & if\ n < d \\ aT\left(\frac{n}{b}\right) + \Theta(n^c) & if\ n \geq d \end{cases}$$

- There are 3 cases

$$if\ c < \log_b a\ \ then\ T(n) = \Theta\left(n^{\log_b a}\right)$$

$$if\ c = \log_b a\ \ then\ T(n) = \Theta(n^c \log n)$$

$$if\ c > \log_b a\ \ then\ T(n) = \Theta(n^c)$$

- Equivalently, comparing b^c to a

The simple form

- Consider mergesort

$$T(n) = \begin{cases} 1 & if\ n < 2 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & if\ n \geq 2 \end{cases}$$

- We determine a, b, and c
  - a = 2
  - b = 2
  - c = 1
- Compare n to $n^{\log_2 2} = n^1$
- (Equivalantly compare 1 to $\log_2 2 = 1$ or 2^1 to 2)

The simple form

- So we are in case 2,
- So

$$T(n) = \Theta(n^c \log_b n)$$

$$= \Theta(n^1 \log_2 n)$$

$$= \Theta(n \log n)$$

The simple form

- From now on we will skip the base case when presenting examples
- Say we have

$$T(n) = T\left(\frac{n}{3}\right) + n$$

- We see
  - a = 1, b = 3, c = 1
- Compare n to $n^{\log_3 1} = n^0 = 1$.
- n is bigger, so so we are in the third case

$$T(n) = \Theta(n^c) = \Theta(n)$$

The simple form

- Say we have

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

- We see
  - a = 4, b = 2, c = 1
- Compare n to $n^{\log_2 4} = n^2$.
- n^2 wins, so we are in the first case

$$T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 4}\right) = \Theta(n^2)$$

The simple form

- Say we have

$$T(n) = 2T\left(\frac{n}{2}\right) + n\log n$$

(e.g. the work at each step involves sorting a list)

- Unfortunately, this equation is not in a format supported by the simple form
- We will need a more general version to handle this
- The more general case will handle

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- For a broader variety of f(n) than the simple version

The Master Theorem

- In the simple version where $f(n) = n^c$ we compared c with $\log_b a$.
- Similarly we will now compare $f(n)$ with $n^{\log_b a}$
- Again three cases

$f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some $\epsilon > 0$ $\quad T(n) = \Theta(n^{\log_b a})$

$f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$ $\quad T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

$f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, $\quad T(n) = \Theta(f(n))$

(and there exists $\delta < 1$ such that $a\, f\left(\frac{n}{b}\right) \leq \delta f(n)$ for all sufficiently large n)

non-examinable

The Master Theorem

- So back to our example $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$
- We have $a = 2, b = 2, f(n) = n \log n$
- $\log_b a = 1$, so we are in case 2 with $k = 1$
- So $T(n) = \Theta(n \log^2 n)$

Proof Sketch (non-examinable)

$$
\begin{aligned}
&T(n) = aT\left(\frac{n}{b}\right) + f(n) \\
&= a\left(aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right) = a^2T\left(\frac{n}{b^2}\right) + a\,f\left(\frac{n}{b}\right) + f(n) \\
&= a^3T\left(\frac{n}{b^3}\right) + a^2f\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\
&= \cdots \\
&= a^{\log_b n}T(1) + \sum_{i=0}^{\log_b n-1} a^i f\left(\frac{n}{b^i}\right) \\
&= n^{\log_b a}T(1) + \sum_{i=0}^{\log_b n-1} a^i f\left(\frac{n}{b^i}\right)
\end{aligned}
$$

- Case 1: $f(n)$ is small, so first term dominates
- Case 3: Second term dominates and is a sum of a geometrically (or faster) decreasing sequence starting with $f(n)$, so proportional to $f(n)$
- Case 2: Second term dominates, and all terms in the sum are proportional to each other, so $f(n)$ times a log factor

(See CLRS Section 4.6 for details)

The Master Theorem in 3 steps

1. Write the recurrence in the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$
2. Compare $f(n)$ to $n^{\log_b a}$.
3. If
   a) $f(n) \gg n^{\log_b a}$ then the residual wins: $T(n) = \Theta(f(n))$
   b) $f(n) \ll n^{\log_b a}$ then the recursive work wins: $T(n) = \Theta(n^{\log_b a})$
   c) $f(n) = \Theta(n^{\log_b a} \log^k n)$ then both matter and we gain a log factor: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Note that it is possible none of these three occurs (how?)

The Master Theorem in 3 steps

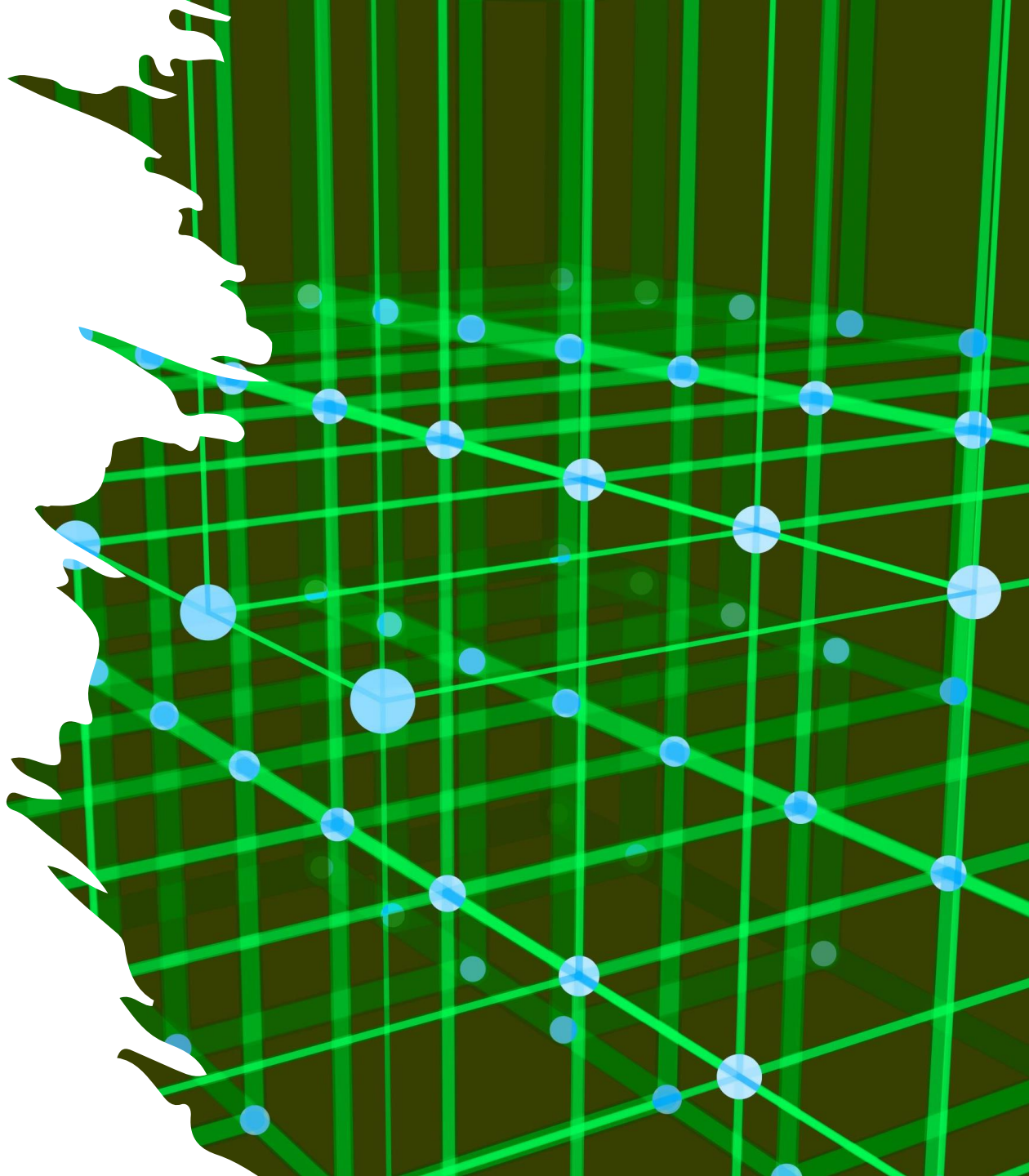
- I wrote $\gg$ and $\ll$ as a way to remember the intuition – what they mean here is *greater than n to a higher power* (and *less than n to a lower power*)
- This is the epsilon thing in the statement of the theorem
- So e.g. $\mathrm{f(n)} = n^{\log_b a} n^{1/n} = n^{\log_b a + 1/n}$ is not case 1
- But $f(n) = n^{\log_b a} \sqrt{n} = n^{\log_b a + 1/2}$ is

Exercises

Goodrich & Tamassia: R-11.1, C-11.3, C-11.4

CLRS: 4.5-1, 4.5-3, 4.5-4, 4-1, 4-2, 4-3

Integer multiplication



The problem

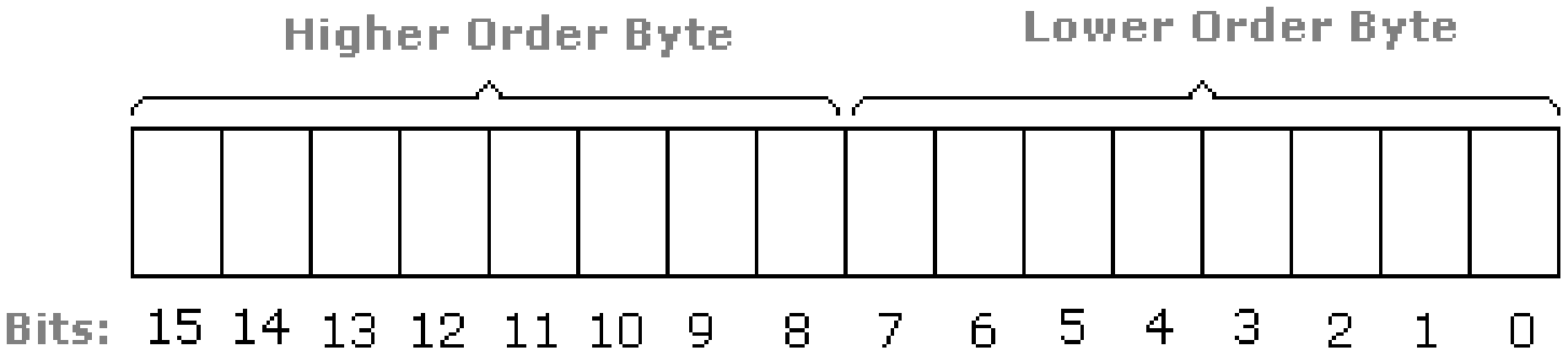
- Processors have a “natural” size for values
  - Generally it depends on how big the registers are, and how wide the on-chip buses are
  - 32- and 64-bit machines are standard right now
  - Called the *word size*
- Sometimes you have to work with integers that require more bits than a machine’s word size
  - This happens a lot in cryptography
- It is important to perform arithmetic operations on these values as quickly as possible, given the need to encrypt/decrypt network communications
- Let us say we have two large integers, I and J
- It is easy to see that computing I + J and I – J can be done in O(n)
  - Where n is the number of bits/digits

The problem

- Computing (I * J) using your grade-school algorithm would be $O(n^2)$
- Can we do better?
- First, some groundwork
- We say bit A is *higher-order* than bit B if bit A corresponds to a higher power of 2
  - Informally, if bit A is to the left of bit B when the number is written down
- We say the same thing about disjoint sequences of bits
  - Sequence A is higher-order than sequence B if all the bits in A are left of all the bits in B
- Informally we say that the left half of a number is its higher-order bits, and the right half are its lower-order bits

The problem

Visually:



The problem

- Let us denote X's higher-order bits by X_H and use X_L for the lower-order bits
- We can then say for I and J:

$$I = I_H 2^{n/2} + I_L$$
$$J = J_H 2^{n/2} + J_L$$

- This allows us to split I and J into halves and work on them separately
- (Note that multiplying by $2^{n/2}$ is just a left-shift)

A first attempt

- Given our definition of high- and low-order bits, we can write

$$I * J = \left(I_H 2^{n/2} + I_L\right)\left(J_H 2^{n/2} + J_L\right)$$

$$= I_H J_H 2^n + I_L J_H 2^{n/2} + I_H J_L 2^{n/2} + I_L J_L$$

- We have divided the number of bits in I and J into half
- There are now four recursive calls on numbers with half as many bits as before
- Plus some additions with cost $\Theta(n)$

A first attempt

- So the recurrence is

$$T(n) = \begin{cases} 1 & if\ n < 2 \\ 4T\left(\frac{n}{2}\right) + n & if\ n \geq 2 \end{cases}$$

- Why?
  - The problem divides into 4 parts
  - Each part is half the size (has half as many bits) as the original
  - Putting the parts back together takes O(n) steps for the additions

A first attempt

- Given this recurrence relation

$$T(n) = \begin{cases} 1 & if\ n < 2 \\ 4T\left(\frac{n}{2}\right) + n & if\ n \geq 2 \end{cases}$$

- Apply the Master Theorem with a = 4, b = 2
- Compare n with $n^{\log_b a} = n^2$
- n^2 wins, so $T(n) = \Theta(n^2)$
- …just like we had before ☹

A second attempt

- If we could cut down a, we could lower the exponent on n
- This would mean reducing the number of recursive calls from 4 to 3 (or fewer)
- We might take advantage of the common bit-shifting in the middle to combine two calls

$$I_H J_H 2^n + (I_L J_H + I_H J_L) 2^{n/2} + I_L J_L$$

- But that does not fix things
  - Putting things in parentheses does not eliminate the need to calculate them
- We need some way to get those two results using only one calculation on n/2 bits

A second attempt

- It turns out that
$$(I_H - I_L)(J_L - J_H) = I_H J_L - I_L J_L - I_H J_H + I_L J_H$$
- It is a single call that operates on only n/2 bits and returns the two values we want
  - $I_L J_H \; and \; I_H J_L$
- Admittedly, they are jumbled together with two values we do not want
  - But we can remove those values with a judicious addition
- This results in
$$I * J = I_H J_H 2^n + [(I_H - I_L)(J_L - J_H) + I_H J_H + I_L J_L]2^{n/2} + I_L J_L$$
- We have only 3 calls now
  - Each only uses half the bits
- Have we cheated here?

A second attempt

- Our problem with the middle term earlier was the need to calculate $(I_L J_H + I_H J_L)$
  - The problem is that is two multiplications
  - We can only have one
- Now we have

$$I * J = I_H J_H 2^n + [(I_H - I_L)(J_L - J_H) + I_H J_H + I_L J_L]2^{n/2} + I_L J_L$$

- Which only needs three multiplications
- It looks like more, but is not
  - We first calculate $I_H J_H, I_L J_L$ and $(I_H - I_L)(J_L - J_H)$ using three multiplies
  - Then we can use these plus some additions and bitshifts

A second attempt

- The new recurrence relation looks like

$$T(n) = \begin{cases} 1 & if\ n < 2 \\ 3T\left(\frac{n}{2}\right) + n & if\ n \geq 2 \end{cases}$$

- Use the Master Theorem with a = 3, b = 2
- Compare n with $n^{\log_b a} = n^{\log_3 2}$

A second attempt

- It is the first case

$$T(n) = \Theta\left(n^{\log_b a}\right)$$
$$= \Theta\left(n^{\log_2 3}\right)$$
$$\approx \Theta(n^{1.585})$$

- This is better than our original – ‘Karatsuba’s algorithm’
- And the Fast Fourier Transform can do even better: $\tilde{O}(n \log n)$

Divide and conquer

- That was an example of solving a problem by breaking it up in a clever way
- This is a common algorithm design paradigm – '**divide and conquer**'
- Steve will look at some more examples on Thursday